SEARCHING
PROBLEM 3

QUESTION:

DS	**Session**	Searching	**Question Information**	Level 1 Challenge 3

Problem Description:
Darsh recently found an new rectangular circuit board that he would like to recycle. That has R rows and C columns of squares.

Each square of the circuit board has a very thinness, measured in millimetres. The square in the r-th row and c-th column has thinness Mr,c. A circuit board is beautiful if in each row, the difference between the thinnest square and the least thin square is no greater than L.

Since the original circuit board might not be beautiful, Darsh would like to find a beautiful subcircuit board. This board can be obtained by choosing an axis-aligned subrectangle from the original board and taking the squares in that subrectangle. Darsh would like your help in finding the number of squares in the largest beautiful subrectangle of his original board.

Constraints:
$1 \le T \le 50$.
$1 \le R \le 205$.
$1 \le C \le 205$.
$0 \le Mi, j \le 10^3$ for all i, j.
$0 \le L \le 10^3$.

Input Format:
The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with one line containing three integers R, C and L, the number of rows, the number of columns, and the maximum difference in thinness allowed in each row.

Then, there are R more lines containing C integers each. The c-th integer on the r-th line is Mr, c, the thinness of the square in the r-th row and c-th column.

Output Format:
Print the output in a separate lines contains the maximum number of squares in a beautiful subrectangle.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
1 4 0
3 1 3 3
2 3 0
4 4 5
7 6 6
4 5 0
2 2 4 4 20
8 3 3 3 12
6 6 3 3 3
1 6 8 6 4
```

EXPECTED OUTPUT

```
2
2
6
```

Test Case 2

INPUT (STDIN)

```
3
2 4 0
2 3 1 2
3 1 3 3
2 3 0
41 4 5
7 61 6
4 5 0
12 12 14 4 20
81 13 13 3 12
61 16 31 3 3
11 16 81 6 4
```

EXPECTED OUTPUT

```
2
2
4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int A[309][309];
```

Test Case 2

KEYWORD

```
bool ok[309][309][309];
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
13	400	53

SOURCE CODE:

```c
#include <stdio.h>

int matrix[305][305];

int main() {
    int num_tests;
    if (scanf("%d", &num_tests) != 1) return 0;

    while (num_tests--) {
        int r_len, c_len, max_diff;
        scanf("%d %d %d", &r_len, &c_len, &max_diff);

        for (int r = 0; r < r_len; r++) {
            for (int c = 0; c < c_len; c++) {
                scanf("%d", &matrix[r][c]);
            }
        }

        int peak_area = 0;
        for (int left = 0; left < c_len; left++) {
            for (int right = left; right < c_len; right++) {
                int consecutive_valid = 0;
                for (int r = 0; r < r_len; r++) {
                    int low = matrix[r][left], high = matrix[r][left];
                    for (int c = left + 1; c <= right; c++) {
                        if (matrix[r][c] < low) low = matrix[r][c];
                        if (matrix[r][c] > high) high = matrix[r][c];
                    }
                    if (high - low <= max_diff) {
                        consecutive_valid++;
                        int current_subarea = consecutive_valid * (right - left + 1);
                        if (current_subarea > peak_area) peak_area = current_subarea;
                    } else {
                        consecutive_valid = 0;
                    }
                }
            }
        }
        printf("%d\n", peak_area);
    }
    return 0;
}
```

OUTPUT:

```
1
4 4 5
1 4 0 2
3 1 3 3
2 3 0 1
7 6 6 4
8
```

```
2
2 4 0
2 3 1 2
3 1 3 3
2 3 0
41 4 5
7 61 6
4 5 0
12 12 14 4 20
81 13 13 3 12
61 16 31 3 3
11 16 81 6 4
4
```